30

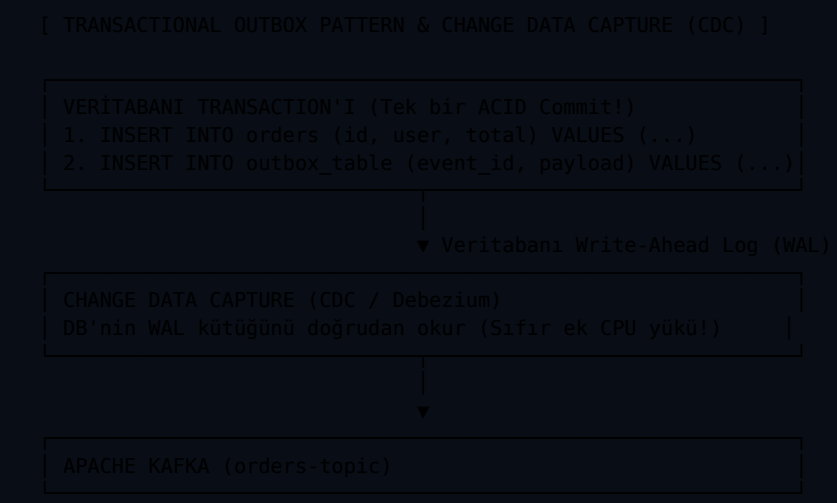
BÖLÜM: ASENKRON İLETIŞIM, MESAJLAŞMA VE OLAY TABANLI MIMARI | MODULE: ASYNCHRONOUS MESSAGING & EVENT-DRIVEN ARCHITECTURE

Transactional Outbox Pattern ve Change Data Capture (CDC / Debezium)

Transactional Outbox Pattern & Change Data Capture (CDC)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirkette sözleşme imzaladığınızı düşünün. Normalde yapılması gereken iki iş vardır: 1) Sözleşmeyi şirket kasasına kilitlemek (DB kaydı), 2) Müşteriye kargo göndermek (Kafka olayı). Eğer sözleşmeyi kasaya koyduktan hemen sonra elektrikler kesilirse ve kargocuya haber veremezseniz sistem bozulur! Transactional Outbox deseni şudur: Kasanın içine küçük bir 'Giden Evrak Kutusu' (Outbox Tablosu) koyarsınız. Sözleşmeyi kilitlerken kargo emrini de AYNI kasanın içine tek bir anahtar çevirmeyle kilitlersiniz (ACID Transaction)! Sonra kargo çırağı (Debezium/CDC) kasadaki kutuyu okur ve kargoyu güvenle yola çıkarır!

🇬🇧 ENGLISH (ELI5)

Imagine signing a commercial contract requiring two atomic actions: 1) Lock the contract in the vault (Database write), 2) Dispatch a courier to the client (Kafka event). If lightning strikes right after locking the vault, the courier is never alerted, corrupting the system! The Transactional Outbox Pattern solves this: Place an 'Outgoing Mailbox' inside the exact same vault! In a single twist of the vault key (one atomic ACID transaction), you lock both the contract and the courier dispatch ticket together! A dedicated courier bot (Debezium / CDC) monitors the outgoing mailbox and dispatches events with zero risk of desynchronization!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

İki Fazlı İki Ayrı Sistem Problemi (Dual-Write Problem): Uygulama önce veritabanına `INSERT` yapıp ardından Kafka'ya `send()` yaparsa; DB başarılı olup Kafka çağrısı çökerse sistem tutarsızlaşır. Transactional Outbox Deseni bu ikili yazma açığını çözer: 1) İş mantığı verisi (`orders`) ve dışarıya fırlatılacak olay (`outbox`) AYNI veritabanında TEK bir yerel ACID transaction içinde commit edilir. 2) Change Data Capture (CDC / Debezium): Veritabanının işlem günlüğünü (PostgreSQL WAL veya MySQL Binlog) doğrudan dinler. Veritabanına hiçbir sorgu atmadan, disk seviyesindeki binary log değişikliklerini yakalayarak Kafka'ya sıfır veri kaybıyla yazar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The Dual-Write Anti-Pattern: A service writes to a relational database and then publishes to Kafka. If the database commits but the network blip crashes the Kafka producer, data desynchronizes permanently. The Transactional Outbox Pattern cures this: 1) Business entities and an `outbox` event table commit together in a single local ACID database transaction. 2) Change Data Capture (CDC / Debezium): An external tailer intercepts the database Write-Ahead Log (WAL / Binlog) directly at the storage engine level with zero query polling overhead, streaming mutations into Kafka with absolute At-Least-Once delivery guarantees.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Transactional Outbox Veri Yapısı ve Atomic Commit Temsili
public class TransactionalOutboxPattern {
    public record OrderRecord(long orderId, String customer, double amount) {}
    public record OutboxMessage(String eventId, String aggregateType, String payload) {}

    // Tek bir yerel ACID veritabanı transaction'ında çalışan metod:
    public void createOrderWithOutbox(OrderRecord order) {
        // BEGIN TRANSACTION;

        // 1. İş tablosuna yaz:
        IO.println("1. DB INSERT INTO orders: ID=" + order.orderId());

        // 2. Outbox tablosuna yaz (Aynı DB Transaction'ı!):
        var outboxEvent = new OutboxMessage(
            "evt_" + System.currentTimeMillis(),
            "ORDER",
            "{"orderId": " + order.orderId() + ", "amount": " + order.amount() + "}"
        );
        IO.println("2. DB INSERT INTO outbox: EventID=" + outboxEvent.eventId());

        // COMMIT TRANSACTION;
        // CDC (Debezium) arka planda WAL'ı okuyup Kafka'ya iletir!
    }
}

void main() {
    var outbox = new TransactionalOutboxPattern();
    outbox.createOrderWithOutbox(new TransactionalOutboxPattern.OrderRecord(9021L, "Ali Bozkurt",
250.0));
    IO.println("✔ Atomik Outbox Kaydı Tamamlandı (Dual-Write Engellendi)");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Outbox tablosunu periyodik olarak `SELECT * FROM outbox WHERE sent = false` şeklinde polling yaparak okumak mülakatta Junior cevabıdır! Tablo büyüdükçe veritabanını kilitler. Senior/Staff seviye cevap: Debezium gibi bir CDC aracının veritabanının binary Write-Ahead Log'unu (WAL) doğrudan dinlemesidir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Proposing a background cron poller (`SELECT * FROM outbox WHERE processed = false`) is an interview red flag that degrades database CPU. The Staff-level answer is transaction log tailing via Change Data Capture (CDC with Debezium reading PostgreSQL WAL).

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Dual-Write problemi, DB ve Kafka'ya aynı anda ayrı ayrı yazmaya çalışmanın felaketidir.
- Outbox tablosu, olayları iş verisiyle aynı ACID transaction'ında güvenceye alır.
- Change Data Capture (CDC / Debezium) veritabanı WAL kütüğünü dinleyerek sıfır yükle çalışır.

🎯 Key Takeaways for the Interview

- The Dual-Write anti-pattern risks fatal desynchronization between DB state and event topics.
- The Outbox table commits domain entities and event envelopes within a single ACID transaction.
- CDC engines (Debezium) tail database WAL logs with zero query polling penalty.